

Practical 5

Aim: Create a registration form with necessary validations using AngularJs.

```
<html ng-app="registrationApp">
```

```
<head>
```

```
  <title>Registration Form</title>
```

```
  <!-- Bootstrap CSS -->
```

```
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
  <!-- AngularJS -->
```

```
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
```

```
<style>
```

```
  body {
```

```
    background-color: #f4f4f4;
```

```
    padding-top: 50px;
```

```
  }
```

```
  .form-container {
```

```
    max-width: 500px;
```

```
    margin: auto;
```

```
  }
```

```
</style>
```

```
</head>

<body ng-controller="RegistrationCtrl">

  <div class="container form-container">

    <div class="card">

      <div class="card-body">

        <h2 class="card-title mb-4">Registration Form</h2>

        <form name="registrationForm" ng-submit="register()" novalidate>

          <!-- First Name -->

          <div class="mb-3">

            <label>First Name:</label>

            <input type="text" name="firstName" ng-model="firstName"
class="form-control" required>

            <span class="text-danger"

              ng-show="registrationForm.firstName.$touched &&
registrationForm.firstName.$error.required">

              First name is required

            </span>

          </div>

          <!-- Last Name -->

          <div class="mb-3">

            <label>Last Name:</label>

            <input type="text" name="lastName" ng-model="lastName"
class="form-control" required>

            <span class="text-danger"
```

```

        ng-show="registrationForm.lastName.$touched &&
registrationForm.lastName.$error.required">

```

```

        Last name is required

```

```

    </span>

```

```

</div>

```

```

<!-- Email -->

```

```

<div class="mb-3">

```

```

    <label>Email:</label>

```

```

        <input type="email" name="email" ng-model="email" class="form-
control" required>

```

```

<span class="text-danger" ng-show="registrationForm.email.$touched &&

```

```

    registrationForm.email.$error.required">

```

```

    Email is required

```

```

    </span>

```

```

<span class="text-danger" ng-show="registrationForm.email.$error.email">

```

```

    Invalid email format

```

```

    </span>

```

```

</div>

```

```

<!-- Password -->

```

```

<div class="mb-3">

```

```

    <label>Password:</label>

```

```

        <input type="password" name="password" ng-model="password"
class="form-control" required>

```

```

        <span class="text-danger"
                ng-show="registrationForm.password.$touched &&
registrationForm.password.$error.required">
                Password is required
        </span>
</div>

```

```

<!-- Phone -->
<div class="mb-3">
    <label>Phone:</label>
    <input type="tel" name="phone" ng-model="phone" class="form-
control" ng-pattern="/^\d{10}$/">
    <span class="text-danger"
            ng-show="registrationForm.phone.$touched &&
registrationForm.phone.$error.pattern">
            Phone must be 10 digits
    </span>
</div>

```

```

<!-- Gender -->
<div class="mb-3">
    <label>Gender:</label>
    <select name="gender" ng-model="gender" class="form-select"
required>
        <option value="">Select Gender</option>

```

```
        <option>Male</option>

        <option>Female</option>

        <option>Other</option>

    </select>

</div>

<!-- Submit -->

        <button type="submit" class="btn btn-success w-100" ng-
disabled="registrationForm.$invalid">

            Register

        </button>

<!-- Success Message -->

<p class="text-success mt-3" ng-show="successMessage">

    {{successMessage}}

</p>

</form>

</div>

</div>

</div>

<script>

    var app = angular.module('registrationApp', []);
```

```
app.controller('RegistrationCtrl', function ($scope) {

    $scope.register = function () {

        if ($scope.registrationForm.$valid) {

            $scope.successMessage = "Registration Successful, Welcome "

                + $scope.firstName + " " + $scope.lastName;

            // Reset form

            $scope.registrationForm.$setPristine();

            $scope.registrationForm.$setUntouched();

            // Clear fields

            $scope.firstName = "";

            $scope.lastName = "";

            $scope.email = "";

            $scope.password = "";

            $scope.phone = "";

            $scope.gender = "";

        }

    };

});

</script>

</body>

</html>
```

Registration Form

First Name:

Last Name:

Email:

Password:

Phone:

Gender:

Registration Form

First Name:

Last Name:

Email:

Password:

Phone:

Gender:

Select Gender 

Register

Registration Successful, Welcome jash baria

Practical 6

Aim: Write a program to create a text file ,update it and read that file and print it using fileSystem module in NodeJs

```
// Import File System module

const fs = require('fs');

console.log("===== ASYNCHRONOUS OPERATIONS =====");

// ASYNCHRONOUS OPERATIONS

function asyncOperations() {

    // 1. CREATE FILE

    fs.writeFile('async.txt', 'Async: First Line\n', (err) => {

        if (err) return console.error(err);

        console.log("Async: File created");

    });

    // 2. UPDATE FILE (APPEND)

    fs.appendFile('async.txt', 'Async: Second Line\n', (err) => {

        if (err) return console.error(err);

        console.log("Async: File updated");

    });

    // 3. READ FILE

    fs.readFile('async.txt', 'utf8', (err, data) => {

        if (err) return console.error(err);
```

```
        console.log("\nAsync File Content:\n" + data);

    });

});

});

}

asyncOperations();

console.log("\n===== SYNCHRONOUS OPERATIONS =====");

// SYNCHRONOUS OPERATIONS

function syncOperations() {

    try {

        // 1. CREATE FILE

        fs.writeFileSync('sync.txt', 'Sync: First Line\n');

        console.log("Sync: File created");

        // 2. UPDATE FILE (APPEND)

        fs.appendFileSync('sync.txt', 'Sync: Second Line\n');

        console.log("Sync: File updated");

        // 3. READ FILE

        const data = fs.readFileSync('sync.txt', 'utf8');

        console.log("\nSync File Content:\n" + data);

    } catch (err) {

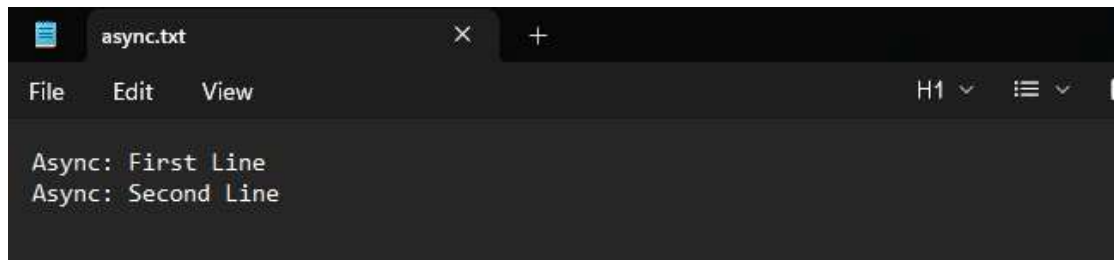
        console.error("Error:", err);

    }

}
```

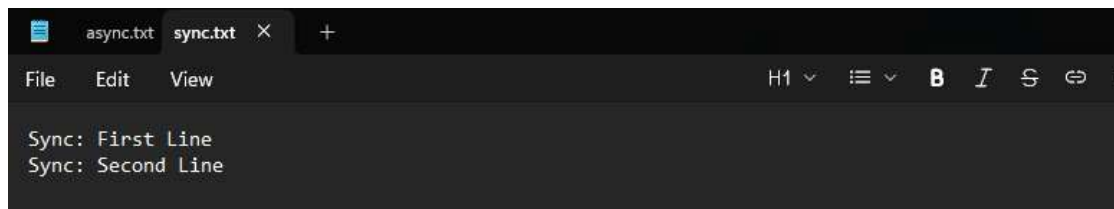
```
}  
  
}
```

syncOperations();



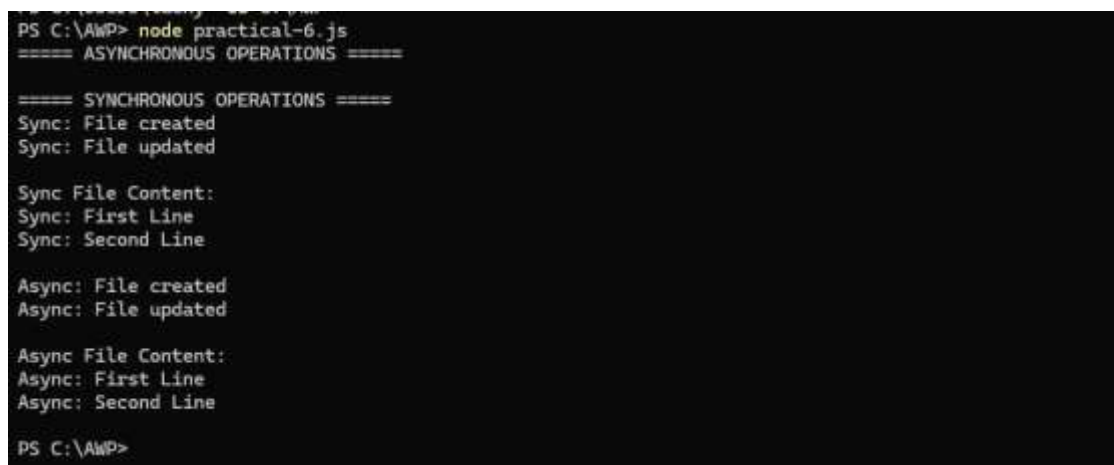
A screenshot of a code editor window titled 'async.txt'. The editor has a dark theme and a menu bar with 'File', 'Edit', and 'View'. The content of the file is:

```
Async: First Line  
Async: Second Line
```



A screenshot of a code editor window titled 'sync.txt'. The editor has a dark theme and a menu bar with 'File', 'Edit', and 'View'. The content of the file is:

```
Sync: First Line  
Sync: Second Line
```



A screenshot of a terminal window showing the output of a Node.js script. The prompt is 'PS C:\AWP> node practical-6.js'. The output is as follows:

```
==== ASYNCHRONOUS OPERATIONS ====  
  
==== SYNCHRONOUS OPERATIONS ====  
Sync: File created  
Sync: File updated  
  
Sync File Content:  
Sync: First Line  
Sync: Second Line  
  
Async: File created  
Async: File updated  
  
Async File Content:  
Async: First Line  
Async: Second Line  
  
PS C:\AWP>
```

Practical 7

Aim: Create a server and load your website on that server using NodeJS.

t.js

```
// Import modules

const http = require('http');
const fs = require('fs');


// Create server

const server = http.createServer((req, res) => {


  // Set default file

  let fileName = 'index.html';


  // Read HTML file

  fs.readFile(fileName, (err, data) => {

    if (err) {

      res.writeHead(404, { 'Content-Type': 'text/plain' });

      res.end("404 Error: File Not Found");

    } else {

      res.writeHead(200, { 'Content-Type': 'text/html' });

      res.end(data); // directly send response

    }

  });

});


// Start server
```

```
server.listen(3000, () => {  
  console.log("Server running at http://localhost:3000");  
});
```



Practical 8

Aim: Write a program to read data from a text file and write in to another text file using streams.

```
// Import fs module
const fs = require('fs');

// File names
const sourceFile = 'input.txt';
const destFile = 'output.txt';

// Create streams
const readStream = fs.createReadStream(sourceFile, { encoding: 'utf8' });
const writeStream = fs.createWriteStream(destFile);

// Pipe data (copy file)
readStream.pipe(writeStream);

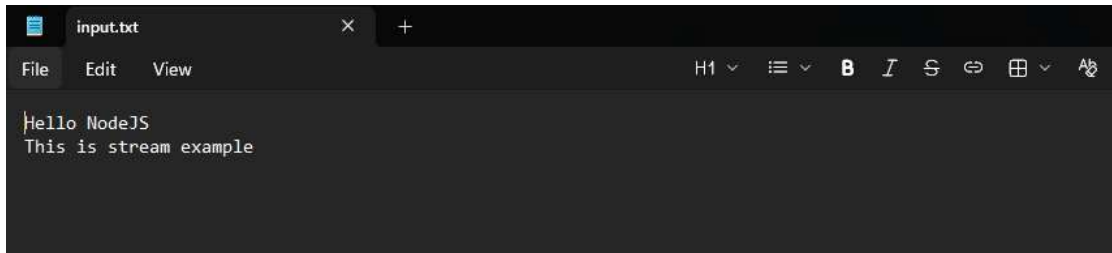
// Success message
writeStream.on('finish', () => {
  console.log("Data copied successfully using streams");
});

// Error handling
readStream.on('error', (err) => {
  console.error("Read Error:", err.message);
});

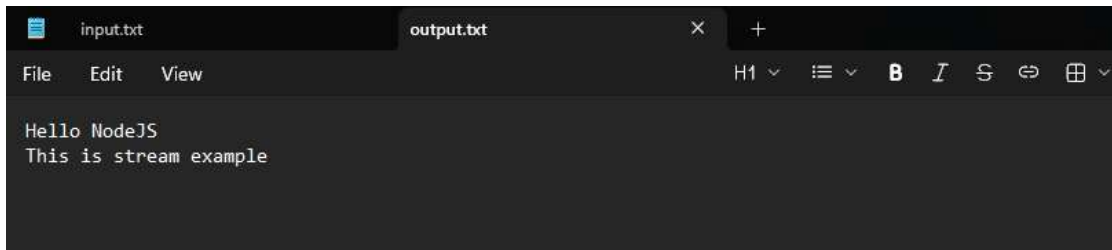
writeStream.on('error', (err) => {
```

```
console.error("Write Error:", err.message);  
});
```

```
Server running at http://localhost:3000  
PS C:\AWP> node practical-8.js  
Data copied successfully using streams  
PS C:\AWP> |
```



A screenshot of a text editor window with a single tab labeled 'input.txt'. The editor has a menu bar with 'File', 'Edit', and 'View'. The right sidebar shows formatting options: 'H1', list icons, bold (B), italic (I), strikethrough (ABC), link, unlink, and a table icon. The main text area contains two lines: 'Hello NodeJS' and 'This is stream example'.



A screenshot of a text editor window with two tabs: 'input.txt' and 'output.txt'. The editor has a menu bar with 'File', 'Edit', and 'View'. The right sidebar shows formatting options: 'H1', list icons, bold (B), italic (I), strikethrough (ABC), link, unlink, and a table icon. The main text area contains two lines: 'Hello NodeJS' and 'This is stream example'.

Practical 9

Aim: Create a NodeJs program to demonstrate session management.

```
// Import modules

const express = require('express');
const session = require('express-session');

const app = express();

// Session configuration
app.use(session({
  secret: 'mysecretkey',
  resave: false,
  saveUninitialized: false, // better practice
  cookie: { maxAge: 60000 } // session valid for 1 minute
}));

// Home route
app.get('/', (req, res) => {
  res.send("Welcome to Home Page<br>Go to /login");
});

// Login route (create session)
app.get('/login', (req, res) => {
  req.session.username = "Lucky";
  res.send("Login successful! <br><a href='/profile'>Go to Profile</a>");
});
```

```
// Profile route (check session)

app.get('/profile', (req, res) => {
  if (req.session.username) {
    res.send(`Welcome ${req.session.username} <br><a
href="/logout">Logout</a>`);
  } else {
    res.send("Session expired or not found. <br><a href="/login">Login</a>");
  }
});

// Logout route (destroy session)

app.get('/logout', (req, res) => {
  req.session.destroy((err) => {
    if (err) {
      return res.send("Error logging out");
    }
    res.send("Logged out successfully");
  });
});

// Start server

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```



```
PS C:\AWP> node practical-9.js
Server running at http://localhost:3000
```





Practical 10

Aim: Create a database in MongoDB and enter the documents in collections.

1. List all databases on the server

Command: show dbs

```
>_MONGOSH
> show dbs
< RoadRescue      132.00 KiB
  StudentDetails  64.00 KiB
  UserDetails     72.00 KiB
  admin           40.00 KiB
  config          108.00 KiB
  joblistiningdb  40.00 KiB
  local           160.00 KiB
test>
```

2. Create / switch to a database

Command: use <databaseName>

Example: use awp

```
> use StudentDetails
< switched to db StudentDetails
StudentDetails>
```

3. Create a collection

Command: db.createCollection(<collectionName>)

Example: db.createCollection("student")

```
> db.createCollection("students")
< { ok: 1 }
```

CRUD Operations

1. Insert Operations

a. insertOne()

Command: `db.collection.insertOne({ <field1>: <value1>, <field2>: <value2>, ... })`

Example: `db.student.insertOne({name:'Aryan',age:20,number:23})`

```
> db.student.insertOne({name: 'Aryan', age: 20, number: 23})
< {
  acknowledged: true,
  insertedId: ObjectId('69f33cdae47c8cb75ee736c2')
}
StudentDetails> |
```

2. Retrieve documents / find All

Command: `db.collection.find({ <query> }, { <projection> })`

Example: `db.student.find()`

```
> db.student.find()
< {
  _id: ObjectId('697f4daf6389b11b74136175'),
  name: 'jash',
  age: 18,
  number: 8
}
{
  _id: ObjectId('697f4df06389b11b74136176'),
  name: 'vedant',
  age: 18,
  number: 9
}
{
  _id: ObjectId('697f50ded24301118d574453'),
  name: 'heet',
  age: 20,
  number: 10,
  _class: 'com.webapp.Model.Student'
}
```

3. Filtered Find

Command: `db.collection.find({ <field>:<value> })`

Example: `db.students.find({ name: "Aryan" })`

```
> db.student.find({ name: "Aryan" })
< {
  _id: ObjectId('69f33cdae47c8cb75ee736c2'),
  name: 'Aryan',
  age: 20,
  number: 23
}
StudentDetails> |
```

4. Update operations

`updateOne()`

Command: `db.collection.updateOne({ <filter> }, { <update> })`

Example: `db.students.updateOne({ name: "Aryan" }, {
$set: { number: 21 } })`

```
> db.students.updateOne( { name: "Aryan" }, {
  $set: { number: 21 } })
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
StudentDetails> |
```

5. Delete operations

deleteOne()

Command: db.collection.deleteOne({ <filter> })

Example: db.students.deleteOne({
name: "Aryan"})

```
> db.students.deleteOne({  
  name: "Aryan"  
})  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
StudentDetails > |
```

Practical 11

Aim: Write a program to connect NodeJS with MongoDB and display the data from the database.

```
const { MongoClient } = require('mongodb');

const url = "mongodb://localhost:27017/";

const dbName = "awp";

const client = new MongoClient(url);

async function connectDB(){
    try {
        await client.connect();
        console.log("Database connected successfully");

        const db = client.db(dbName);

        const collection = db.collection("students");

        const result = await collection.insertOne({
            name: "Jash",
            age: 21,
            branch: "IT"
        });
    }
}
```

```
    console.log("Data inserted: " + result.insertedId);

    const data = await collection.find().toArray();

    console.log("All records: ");

    console.log(data);

  } catch {

    console.log("ERROR: " + err);

  } finally {

    await client.close();

  }

}

connectDB();
```

A terminal window with a black background and white text. It shows the execution of a Node.js script named 'mongodb.js'. The output includes a confirmation of database connection, the insertion of a record with a specific ID, and a list of all records in the collection, which contains one object with fields for _id, name, age, and branch.

```
PS C:\AWP> node mongodb.js
Database connected successfully
Data inserted: 69f39862273d51586d227b88
All records:
[
  {
    _id: new ObjectId('69f39862273d51586d227b88'),
    name: 'Jash',
    age: 21,
    branch: 'IT'
  }
]
PS C:\AWP>
```